

Adaptive - Model-Agnostic LLM Cost Optimizer

Minimum capable intelligence for every request - profiled, routed, verified, measured.

Adaptive is a multi-tenant LLM gateway built with FastAPI (Python 3.10) and a React (Vite) control-center dashboard. Customers obtain a gateway API key, connect their OWN providers (base_url + API key + model_id), and route requests through the gateway with model="auto". The system auto-profiles every connected model's capabilities (customers never label models cheap/mid/frontier), understands every request, selects the cheapest capable model from the customer's own pool, caches reusable context (namespaced per customer), verifies answer quality, escalates when necessary, and measures the actual cost/quality trade-off against an always-best-model baseline - net of control-plane overhead.

```
PROFILE -> UNDERSTAND -> FILTER -> OPTIMIZE -> GENERATE -> VERIFY -> ESCALATE -> MEASURE -> LEARN
```

1. The Problem

Every request sent to the strongest (most expensive) model wastes money; every request sent to a weak model risks quality. Cost savings claimed without quality measurement are not optimization. Teams also waste time manually labeling models as cheap/mid/frontier, and AI-assisted routing often hides the cost of the routing AI itself inside the reported savings.

2. The Solution

- Gateway tenancy - customers get a gw_ API key (encrypted, shown once) and register their own models (base_url + encrypted credential + model_id).
- Model Registry - customers connect any models; no cheap/frontier labels are ever required.
- Model Profiler - measures per-category capability scores on a 24-item test set, through the customer's own endpoint. Scores are benchmark performance on our set, never claimed as universal intelligence scores.
- Task Analyzer - transparent heuristics: task type, difficulty 0-1, confidence, required capabilities + thresholds.
- Hybrid Router - Nemotron (control plane) RECOMMENDS a model; a deterministic validator DECIDES (exists / owner / enabled / context fits / capabilities / pricing valid / not blocked), falling back to the deterministic router. Provenance is always reported: nemotron_recommended | deterministic | nemotron_rejected_fallback.
- Dynamic tiers - cheap/mid/frontier derived from the customer's own pool (price terciles), never hard-coded.
- Quality Evaluator - deterministic, zero extra LLM calls: $0.5 \times \text{correctness} + 0.3 \times \text{relevance} + 0.2 \times \text{completeness}$, labeled reference (grounded) or estimated (heuristic, never ground truth).
- Escalation - quality below threshold retries the next-best model (capped attempts, honest summed cost), preferring in-tier models and only reaching above-baseline-tier models as a last resort.
- Prompt cache - in-memory, namespaced per customer: exact-prompt hits skip the LLM (savings measured); same-context/new-question hits count avoided tokens (savings estimated). Never conflated. Tenant A can never hit tenant B's entries.
- Cost engine - actual spend + counterfactual always-best baseline (measured tokens x best-model pricing; no duplicate expensive calls). $\text{net_savings_usd} = \text{gross savings} - \text{control-plane overhead}$.
- Pricing integrity - declared prices win, then the curated catalog, then pricing_status "unknown" - prices are NEVER fabricated; unpriced models are excluded from cost-optimal routing (clean 400, not a guess).
- Token optimizer - free, deterministic: prompt normalization (code fences preserved), chars/4 token estimation, and predicted output budgets (128/256/512) so short answers don't pay for a 512-token allowance. Cache hits are checked BEFORE the LLM classifier.
- Benchmark Lab - 50 reference-scored queries across 10 categories; baseline quality measured on a deterministic n=5 sample. Plus a naive-vs-optimized token-efficiency benchmark.

3. The Core Pipeline

Every request flows through nine stages. Cache is checked first (before any LLM or control-plane call), so a hit avoids even the classifier call.

```
Request (model=auto)
|
v
[1] Cache check  exact -> semantic -> context      (hit = $0 spend, MEASURED savings)
| miss
v
[2] Task Analyzer / Nemotron classifier              (type, difficulty, capabilities)
v
[3] Hybrid Router  Nemotron RECOMMENDS -> deterministic validator DECIDES
|                                                         (owner/enabled/context/caps/pricing)
v
[4] Token optimizer  normalization + output budget (free, deterministic)
v
[5] GENERATE via the CUSTOMER's own endpoint          (their base_url + their key)
v
[6] VERIFY quality  0.5*correct + 0.3*relevance + 0.2*completeness
| fail
v
[7] ESCALATE to next-best model (capped attempts, summed cost)
v
[8] MEASURE  actual vs always-best baseline, net of control-plane cost
v
[9] LEARN  routing stats + request history -> dashboard; re-profile to refresh
```

4. Architecture

```
React (Vite) --REST--> FastAPI --> Optimizer --+--> Cache check
                                                +--> Control plane (Nemotron via OpenCode Zen)
                                                |   +- classifier (task type/difficulty)
                                                |   +- cache verifier (veto-only)
                                                |   +- LLM evaluator (subjective only)
+--> Task Analyzer -> Hybrid Router
+--> Customer's provider endpoint
+--> Quality -> Escalate
+--> Cost + baseline engine
+--> MongoDB (Atlas) / memory fallback
```

Backend layout (backend/)

- core/ - optimizer, router, cache, quality, profiler, registries, cost engine, token optimizer.
- api/ - v1 gateway (multi-tenant) + platform control-center endpoints.
- providers/ - adapter hierarchy: BaseLLMProvider -> OpenAICompatibleProvider | OpenAIProvider | CustomProvider.
- llm/ - control plane: classifier, cache verifier, LLM evaluator, per-request cost ledger.
- database/ - MongoDB (Atlas) analytics store with in-memory fallback.
- benchmark/ - 50-query reference-scored dataset + runner + token-efficiency benchmark.
- data/ - pricing registry, model profiles, capability tests (JSON, single-writer MVP).

Frontend layout (frontend/)

- Command Center - KPIs, cost comparison, request flow, model distribution, quality gauge.
- Models - registry table with pricing status, capability source, profile status.
- Playground - run the full pipeline with a prompt, context, and reference answer.
- Cache - hit rates, avoided tokens, measured vs estimated savings.
- Benchmark Lab - run benchmarks, mode selector, latest stored result.

5. Gateway Tenancy (Multi-Tenant)

- POST /v1/customers provisions a customer and returns a gw_... API key exactly once (XOR-encrypted at rest, never logged or echoed back).
- Every /v1 request authenticates with Authorization: Bearer gw_...; missing/invalid keys get clean 401s.
- Tenant isolation is verified live: customer B sees 0 models, gets 404s on A's models, and 0 analytics rows.
- Customer credentials are decrypted only per-call, inside the provider factory; the platform's OpenCode key is never used for customer traffic.

6. Hybrid Routing - AI Recommends, Code Decides

The control plane uses Nemotron 3.5 Lightning Free (default model id nemotron-3.5-lightning-free, env-configurable via OPENCODE_MODEL) through OpenCode Zen. It never decides alone:

1. RECOMMEND - Nemotron sees the task analysis and the customer's candidate pool and proposes a model.
2. VALIDATE - the deterministic router checks: model exists, belongs to this customer, is enabled, context fits the window, required capabilities are met, pricing is valid, model not blocked.
3. FALLBACK - any failed check falls back to the deterministic router; provenance (nemotron_recommended | deterministic | nemotron_rejected_fallback) is always reported in the response.

Control-plane jobs: classifier (task type/difficulty as strict JSON, falls back to the legacy analyzer on any failure), cache verifier (veto-only - it can never approve a reuse the deterministic gates blocked), and LLM evaluator (subjective tasks only, labeled llm_judge). Every control-plane token is priced in a per-request ledger.

7. Cache - Namespaced Per Customer

Tier	Match	Savings label	Notes
Exact	SHA-256 of normalized prompt	MEASURED	\$0 spend; skips LLM and classifier
Semantic	canonicalized char-3gram cosine + safety gates	MEASURED	veto-only LLM verifier may block reuse
Context	same context, new question	ESTIMATED	counts avoided input tokens

Only quality-passed answers are stored. Tenant A can never hit tenant B's entries (namespace = customer_id). Cache is checked BEFORE the LLM classifier, so a hit avoids even the control-plane call (classifier_calls_avoided_exact/semantic).

8. Honest Cost Accounting

Field	Meaning
actual_cost_usd	What was really spent (summed across escalation attempts)

baseline_cost_usd	What the always-best model would have cost (measured tokens x best-model price)
savings_usd	baseline - actual (gross savings)
control_plane_cost_usd	What the AI routing itself cost (classifier/verifier/evaluator tokens)
net_savings_usd	gross savings - control-plane overhead; AI never hides its own cost

Usage is provider-reported when available, otherwise estimated at chars/4 and flagged usage_estimated - never fabricated. Pricing resolution: customer-declared -> curated catalog -> pricing_status "unknown" (excluded from cost-optimal routing with a clean 400).

9. API Surface

Gateway API (/v1, multi-tenant - auth: Authorization: Bearer gw_...)

Method	Path	Notes
POST	/v1/customers	provision customer; gw_ key returned exactly once
GET	/v1/customers/me	whoami
POST	/v1/models/register	connect the customer's own model (base_url + key + prices)
GET	/v1/models	my models (?enabled_only=true)
GET/PUT/DELETE	/v1/models/{id}	read / update (rotate key, reprice) / delete
POST	/v1/models/{id}/test	live connectivity test through the customer's endpoint
POST	/v1/models/{id}/profile	auto-profile capability (background job)
POST	/v1/optimize	full pipeline over the customer's pool
POST	/v1/chat/completions	OpenAI-compatible; model="auto" = gateway routes
GET	/v1/analytics	customer-scoped analytics + routing stats
GET	/v1/health	gateway health for the caller

Platform API (/api, single-tenant control center)

- /health, /api/chat (full pipeline), /api/route/preview (free dry-run, no LLM call).
- /api/models CRUD + /api/models/profiles + profiling jobs (start + poll progress).
- /api/models/control-plane - CP config, budgets, health, lifetime stats.
- /api/analytics, /api/routing-stats, /api/requests/{id}, /api/cache/stats, /api/cache/clear.
- /api/benchmark/queries | run | jobs/{id} | latest; /api/benchmark/token-efficiency (+jobs).
- /api/test/generate - single-model smoke test.

10. Verification Status (All Green)

Suite	Result
11 unit/integration suites (registry, control-plane, router, quality, cache+cost, profiler, token-optimizer, benchmark, audit-fixes, gateway-core, v1-api)	ALL exit 0
79-check offline /v1 API suite (test_v1_api.py, no network)	PASS
106-check LIVE smoke (real OpenCode generation: live ping, 24-test profiling job, optimize quality 0.9, exact cache hit, forced-model + escalation, tenant isolation, net-savings analytics)	106/106 PASS

11. Quickstart

```
cd llm-cost-optimizer
Copy-Item .env.example .env      # set OPENCODE_API_KEY (https://opencode.ai/auth)
pip install -r backend\requirements.txt
uvicorn backend.main:app --reload --port 8000

# Frontend
cd frontend; npm install; npm run dev      # http://localhost:5173 (proxies /api -> :8000)
```

Gateway in 60 seconds (PowerShell)

```
$r = Invoke-RestMethod -Method Post -Uri http://localhost:8000/v1/customers `
    -ContentType "application/json" -Body '{"customer_id":"acme","name":"Acme Corp"}'
$H = @{ Authorization = "Bearer $($r.api_key)" }      # shown ONCE - store it

Invoke-RestMethod -Method Post -Uri http://localhost:8000/v1/models/register -Headers $H `
    -ContentType "application/json" -Body (@{ model_id="acme-mini"
    base_url="https://api.acme.ai/v1"; api_key="sk-acme-1"
    input_per_1M=0.12; output_per_1M=0.48 } | ConvertTo-Json)

Invoke-RestMethod -Method Post -Uri http://localhost:8000/v1/chat/completions `
    -Headers $H -ContentType "application/json" -Body (@{ model="auto"
    messages=@(@{role="user"; content="What is an API?"}) } | ConvertTo-Json -Depth 5)
```

12. Honest Limitations

- Quality scoring is lexical, not semantic; the strict 0.75 bar drives high escalation on terse answers.
- Capability profiles use 4 samples/category - noisy; the UI labels measured vs estimated everywhere.
- In-memory cache + JSON registries are single-process (fine for a hackathon, not multi-replica prod).
- Customer-supplied base_url is an accepted SSRF surface for the MVP (documented in SECURITY.md); production needs allow-lists/egress controls.
- Credential encryption is XOR-keystream obfuscation, not KMS-grade.
- No per-customer rate limits/quotas yet.
- Baseline quality from an n=5 sample - reported with n, not hidden.
- The LLM evaluator is a judge, not ground truth; subjective scores are labeled llm_judge.

13. Future Improvements

- Semantic (embedding) similarity for the cache tier.
- More profiler items per category; scheduled re-profiling from live outcomes (LEARN step is currently manual re-run).
- Provider-side prompt caching wired to measured cache-read billing.
- Per-model latency tracking in routing; budget-constrained routing mode.
- Per-customer rate limits, quotas, and key rotation endpoints.
- KMS-backed credential storage; audit log for security events.
- Aggregate control-plane spend in analytics (currently per-request ledger + benchmark totals).

14. Project Documentation

File	Contents
------	----------

API.md	Full /v1 + /api endpoint reference with request/response shapes
MODEL_REGISTRY.md	Registration, pricing statuses, profiling, capability sources
ROUTING.md	Hybrid router: recommendation, validation gates, fallback, provenance
SECURITY.md	Key handling, credential encryption, SSRF caveat, threat model
DEMO_GUIDE.md	Step-by-step live demo walkthrough for judges
ARCHITECTURE.md	Component diagram, data flow, module responsibilities
README.md	Overview, setup, env vars, endpoint tables, methodology